

HAF-EPA Model Development, Training Process, And Justification for Machine Learning

Step 1: Collect Raw Data

Employee Dataset = 2,500

Project Dataset = 900

Step 2: Data Preprocessing/ Normalize

Missing values handle/Duplicate data remove/ Skills normalize etc.

Step 3: Employee–Project Pair Generation (process/pair_creation)

A single employee or a single project alone does not provide enough information for the model to learn suitability. The model must evaluate the interaction between employee characteristics and project requirements. For this reason, an employee-project pair is created for every possible combination. A pair is created between each employee and each project, resulting in a comprehensive employee project pairing dataset.

Example:

Christopher + Project 1,

Christopher + Project 2

.....

Christopher + Project 900

Calculation: $2500 \times 900 = 2,250,000$ pairs

Step 4: Feature Engineering (process/feature_engineering)

Machine learning models cannot effectively understand raw employee and project information. Therefore, the raw data must first be transformed into meaningful numerical features that represent the relationship between an employee and a project.

Feature engineering converts employee characteristics and project requirements into measurable compatibility scores. These scores provide the machine learning model with a structured representation of how well an employee matches a project.

Without feature engineering, the model would only see raw attributes such as skills, experience, availability, and project requirements, making it difficult to learn meaningful suitability patterns.

Each employee project pair is processed to calculate the corresponding feature set.

Features:

matched_skill_count	required_skill_count	strong_skill_match
matched_required_skill_count	skill_match_score	weighted_skill_match_score
employee_skill_count	employee_skill_coverage	avg_experience_on_required_skills
project_skill_count	missing_required_skill_count	availability_fit_score
		
		Total : 19 Features

Example:

Raw Data:

Employee:

- Skills: Python, SQL, Machine Learning
- Experience: 4 years
- Availability: 60%

Project:

- Required Skills: Python, SQL, Docker
- Required Experience: 5 years

Feature Engineering converts this into:

skill_compatibility_score = 0.67
experience_alignment_score = 0.80
availability_score = 0.60
.....

These numerical values are much easier for the Random Forest model to analyze and learn from.

Step 5: Create Labels (process/labeling)

Labels are required because HAF-EPA uses supervised machine learning, where the model learns the relationship between employee-project features and known suitability outcomes (Suitable = 1, Not Suitable = 0) to predict future project allocations.

Each employee–project pair is labeled based on its suitability for project allocation:

Suitable (1) – The employee is considered a suitable match for the project.

Not Suitable (0) – The employee is not considered a suitable match for the project.

Example:

Employee	Project	Label
Christopher	Project A	Suitable (1)
Christopher	Project B	Not Suitable (0)
Donna	Project C	Suitable (1)

Step 6: Create Final Training Dataset

Machine learning algorithms require the input data and the corresponding target values to be organized in a structured format before training.

After feature engineering, we have $\Rightarrow$ Feature Matrix (X)

And after labeling, we have $\Rightarrow$ Labels (Y)

The final training dataset combines these two components so that each employee-project feature vector is associated with its correct suitability label.

$\Rightarrow$ **Feature Matrix (X) + Labels (Y)**

Example:

skill_match	experience	availability	Label
1.00	12.4	0.30	1
0.20	1.5	0.10	0
0.90	8.5	0.70	1

$\Rightarrow$ Feature Matrix (X) + Labels (Y)

Feature Matrix (X)

```
[  
  [1.00, 12.4, 0.80],  
  [0.20, 1.5, 0.10],  
  [0.90, 8.5, 0.70]  
]
```

Labels (Y)

```
[1, 0, 1]
```

Final Training Dataset = X + Y

Step 7: Train-Test Split (pipeline/split)

The training dataset is used to teach the Random Forest model, while the testing dataset is used to evaluate how well the trained model performs on previously unseen employee-project pairs.

If the model is evaluated using the same data it was trained on, it may simply memorize patterns instead of learning general rules. As a result, the performance would appear unrealistically high, that overfit.

By reserving a separate testing dataset, we can measure how well the model performs on new employee-project pairs that it has never seen before.

Total Employee-Project Pairs = 2,250,000

⇒ Training Data (80%) = 1,800,000 pairs

⇒ Testing Data (20%) = 450,000 pairs

Step 8: Handle Class Imbalance (pipeline/balance)

Before training, the dataset contains a severe class imbalance:

Suitable (1) = 4,716 pairs

Not Suitable (0) = 1,795,284 pairs

If the model is trained on the original dataset, it may learn to predict "**Not Suitable**" for almost every employee-project pair. Such a model would fail to identify suitable employees, making it useless for employee recommendation.

To address this problem, undersampling is applied.

The majority class (Not Suitable) is reduced so that both classes contain an equal number of examples.

Suitable = 50%

Not Suitable = 50%

This balanced dataset allows the Random Forest model to learn patterns from both suitable and not suitable employee-project pairs.

Step 9: Train Random Forest Model (models/train_model.py)

In this step, the prepared employee-project pair dataset is used to train the machine learning model.

Each row represents one employee-project combination, and the model learns from the engineered features such as skill compatibility, experience alignment, availability, past performance, collaboration score, and task-context match.

The target label is:

1 = Suitable employee for the project

0 = Not suitable employee for the project

The project uses a **Random Forest Classifier**. Random Forest works by creating many decision trees during training. Each tree learns different patterns from the data, and during prediction, the final result is calculated by combining the decisions of all trees.

Example configuration of the Random Forest model:

```
RandomForestClassifier(  
    n_estimators=300,  
    max_depth=14,  
    class_weight="balanced",  
    random_state=42,  
    n_jobs=-1  
)
```

Where:

N_estimators = 300 → 300 decision trees are created.
Max_depth = 14 → each tree can go a maximum of 14 levels deep.
Class_weight = "balanced" → helps the model handle imbalance between suitable and not suitable pairs.
Random_state = 42 → keeps the result reproducible.
N_jobs = -1 → uses all available CPU cores for faster training.

After configuring the model, the feature matrix **X_train** and the target labels **y_train** are passed to the model for training:

```
model.fit(X_train, y_train)
```

During training, the Random Forest model learns the relationship between the employee-project feature values and the suitability labels.

For example, it learns which combinations of skill match, experience, availability, performance, and collaboration usually lead to a suitable project allocation.

After training, the model does not need to manually check every rule again. Instead, it stores the learned decision patterns inside the trained model file.

The trained model is saved as: **HAF-EPA.joblib**

This saved model is later loaded during the prediction phase to evaluate new employee-project pairs and estimate their suitability probabilities.

Step 10: What Happens During Training

This is one of the most important stages of the HAF-EPA framework because this is where the machine learning model learns how to distinguish suitable employee-project assignments from unsuitable assignments.

The Random Forest algorithm creates **300 decision trees** because the parameter **n_estimators=300** is used. Each decision tree learns patterns from the training dataset by analyzing the relationship between employee project features and the target label (**Suitable or Not Suitable**).

For example, the model how learn patterns such as:

⇒ **One decision tree may learn the following rule:**

IF
skill_compatibility_score > 0.80
AND
availability_score > 0.40
THEN
Suitable

⇒ **Second decision tree may learn a different rule:**

IF
missing_required_skill_count > 2
THEN
Not Suitable

⇒ **Third decision tree may learn :**

IF
experience_alignment_score > 0.75
AND
past_performance_score > 0.85
THEN
Suitable

Each tree learns different patterns because Random Forest trains every tree on a different subset of the training data and a different subset of features.

As a result, the trees do not all make exactly the same decisions. This diversity helps improve the robustness and accuracy of the model.

How the Final Decision is Made

When a new employee-project pair is provided, all 300 decision trees make their own prediction.

Example:

Tree 1 → Suitable
Tree 2 → Suitable
Tree 3 → Not Suitable
.....
Tree 300 → Suitable

The final prediction is determined by **majority voting**:

$255 / 300 = 85\%$

Final Prediction: Suitable

Suitability Probability: 85%

As a result, the trained HAF-EPA model captures knowledge from historical employee-project assignments and stores this learned knowledge inside the trained model file:

Example, How it work:

A simple analogy is teaching a child how to identify fruits.

Initially, the child is shown many examples of apples, bananas, and oranges.

Over time, the child learns the characteristics of each fruit. Later, when presented with a new fruit, the child does not need to relearn everything from the beginning; instead, the child uses previous knowledge to make a decision.

In the same way, the Random Forest model learns from historical employee-project matches during training. When a new project arrives, the model applies its learned knowledge to predict which employees are most suitable for that project.

Step 11: Save Trained Model

After the training process is completed, the learned Random Forest model is saved as:

HAF-EPA.joblib

This file contains all the knowledge learned during training, including the decision patterns and relationships between employee-project features and suitability outcomes.

Saving the trained model eliminates the need to retrain the system every time a new project is submitted.

Instead, when a new project arrives, the **HAF-EPA** framework simply loads the previously trained model and uses the learned knowledge to evaluate employee-project pairs.

The saved model acts as the knowledge base of the HAF-EPA framework.

During the prediction phase, the framework loads **HAF-EPA.joblib**, generates employee-project feature vectors for the new project, and predicts suitability scores without repeating the training process.

Therefore, the purpose of this step is to preserve the learned intelligence of the HAF-EPA model so that it can be efficiently reused for future employee allocation recommendations.

From Training to Real-World Usage

Up to this point, the HAF-EPA model has been trained using historical employee-project assignments and saved as ***HAF-EPA.joblib***.

During training, the model learned patterns from millions of employee-project pairs and discovered which combinations of skills, experience, availability, performance, and collaboration characteristics are typically associated with successful project assignments.

However, the purpose of HAF-EPA is not only to train a machine learning model but also to support real-world employee allocation decisions.

Once the training process is complete, the trained model can be used to evaluate completely new projects that were not part of the training dataset.

In a real-world scenario, a project manager creates a new project and uploads a project description document (PDF) through the web application. The PDF may contain project requirements such as required skills, project objectives, technical requirements, business requirements, responsibilities, and project context.

The web application automatically analyzes the uploaded project, extracts the relevant information, generates employee-project features, and uses the trained HAF-EPA model to predict which employees are most suitable for the project.

The following steps demonstrate how the trained model is applied to a new project within the web application.

Step 1: Prediction Phase (Applied in the Web Application)

The prediction process begins when a project manager uploads a new project description document through the web application.

Example: Upload File → StockPro.pdf

The uploaded PDF contains information about the project requirements. These requirements may include:

- Required skills
- Optional skills
- Project objectives
- Technical requirements
- Business requirements
- Project responsibilities
- Project domain and context

Example ⇒ **project requirements extracted from StockPro.pdf:**

Required Skills:

- Data Analysis
- SQL
- Testing

Project Domain:

- Inventory Management System

Project Goal:

- Develop and test a stock management application

The purpose of uploading the project document is to provide the system with information about the project so that HAF-EPA can identify employees whose skills, experience, and availability best match the project requirements.

Step 2: Extract Skills from the New Project PDF

After the project document is uploaded, the system reads and analyzes the PDF content to identify project requirements.

The extracted information may include:

- Required skills
- Optional skills
- Project domain
- Project responsibilities
- Technical requirements

The machine learning model cannot directly understand a PDF document. Therefore, the project requirements must first be extracted and converted into structured information that can be compared with employee skills and experience.

Step 3 : Generate Employee Features

After extracting the project requirements, HAF-EPA compares the project with each employee profile and generates a set of numerical features.

These features describe how well an employee matches the requirements of the project.

For example, consider the employee **Christopher**.

matched_required_skill_count	= 3
project_skill_count	= 3
skill_match_score	= 1.0
employee_skill_coverage	= 0.333
availability_fit_score	= 0.30
avg_experience_on_required_skills	= 12.42

From Christopher Skills list it matched 3 skills

⇒ matched_required_skill_count = 3

Step 4: Apply the Trained HAF-EPA Model

After generating the employee-project features, the feature vector is passed to the trained HAF-EPA Random Forest model.

Example for Christopher:

Feature Vector $\Rightarrow$ [3, 3, 1.00, 0.333, 0.30, 12.42, ...]

The Random Forest model does not manually compare Christopher with the project requirements. Instead, it uses the knowledge learned during training from historical employee-project assignments.

Step 5: Load Trained Model (HAF-EPA.joblib)

During the training phase, the Random Forest model learned patterns from historical employee-project assignments. These learned patterns are stored in the trained model file HAF-EPA.joblib.

Therefore, during prediction, the model does not need to be trained again. Instead, the previously trained model is loaded and reused.

The loaded model contains the knowledge acquired from millions of employee-project pairs and can immediately evaluate new employee-project feature vectors.

Similar to how a human uses previous experience to make decisions, the trained model uses its learned knowledge to estimate the suitability of employees for a new project without repeating the entire training process.

Example: $\Rightarrow$ Load **HAF-EPA.joblib**

Step 6: Apply the HAF-EPA Model to Predict Employee Suitability

At this stage, the feature vector for the new project has been generated, and the trained HAF-EPA model (**HAF-EPA.joblib**) has already been loaded.

The generated feature vector is passed to the trained Random Forest model.

Using the knowledge learned from historical employee-project assignments, the model evaluates the feature values and estimates the probability that the employee is suitable for the project.

This feature vector is generated from the uploaded project and employee information.

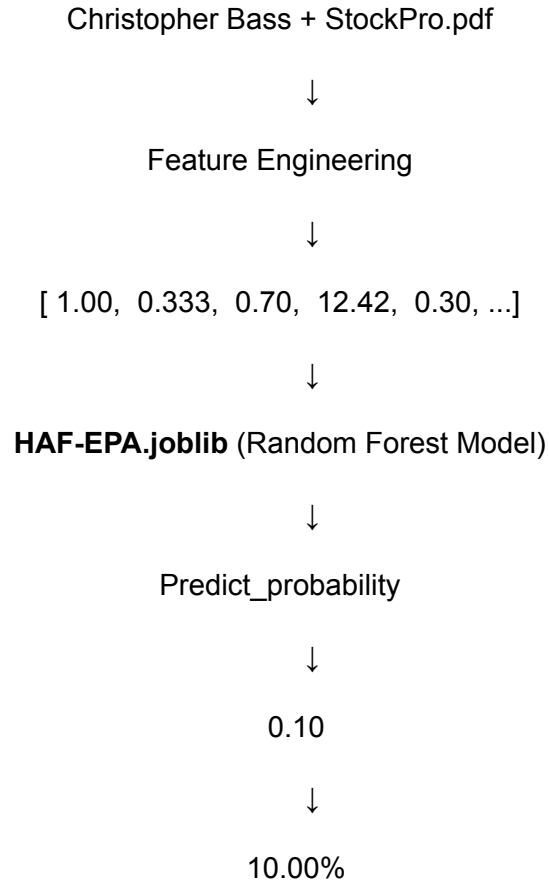
For example, consider the employee Christopher Bass.

```
[
    1.00, # skill_match_score
    0.333, # employee_skill_coverage
    0.70, # weighted_skill_match_score
    12.42, # avg_experience_on_required_skills
    0.30, # availability_fit_score
    1.00, # strong_skill_match
    0.00, # missing_required_skill_count
    3.00, # matched_required_skill_count
    3.00, # project_skill_count
    9.00, # employee_skill_count
    ...
]
```

The model analyses the complete feature vector and estimates the probability that the employee-project pair belongs to the **Suitable (1)** class.

As output, the model returns a suitability probability.

For example:



The same process is repeated for every employee. Finally, all employees are ranked according to their predicted suitability scores, and the highest-ranked candidates are recommended for the project.

Question and Answer

1. Why do we need Machine Learning , when we already know all employee skills?

To answer this question, let's look at our new project described in "**StockPro.pdf**".

In this project, we have three existing employees, and all **three employees** match all **three required skills**.

In that case, simply checking the skills is not enough because all employees appear equally qualified.

Also, in a real-world company, it is not practical to manually check every condition, compare every employee, and analyze who is the best match for each new project. As the number of employees and projects increases, this process becomes very difficult and time-consuming.

Machine Learning helps by automatically analyzing many factors such as skills, experience, availability, past performance, and historical project assignments.

⇒ **Using traditional rule-based programming**, all three employees would receive exactly the same result because all of them satisfy the required project skills. This is where Machine Learning becomes necessary.

Employee suitability does not depend only on skill matching. It also depends on the combination of multiple factors such as:

- Skill compatibility
- Experience alignment
- Availability
- Historical performance
- Employee skill coverage
- Workforce characteristics
- Other engineered project-allocation features

In HAF-EPA, these factors are converted into a numerical feature vector and evaluated by the trained Random Forest model.

For example, Christopher Bass is represented as:

```
[  
    1.00, # skill_match_score  
    0.333, # employee_skill_coverage  
    0.70, # weighted_skill_match_score  
    12.42, # avg_experience_on_required_skills  
    0.30, # availability_fit_score  
    1.00, # strong_skill_match  
    0.00, # missing_required_skill_count  
    3.00, # matched_required_skill_count  
    3.00, # project_skill_count  
    9.00, # employee_skill_count  
    ...  
]
```

The same process is performed for James Collins and Donna Jordan.

The Random Forest model then evaluates the complete employee-project feature profile and predicts a suitability score for each candidate.

Predicted Results:

Employee	Project Match	ML Suitability
Christopher Bass	100%	10.00%
James Collins	100%	9.00%
Donna Jordan	100%	8.67%

Although all three employees achieved the same skill-matching result, the Machine Learning model identified Christopher Bass as the most suitable candidate based on the complete feature profile learned from historical employee-project assignments.

Therefore, Machine Learning is not used to determine whether an employee has the required skills.

Machine Learning is used to determine which employee is the most suitable among multiple employees who already satisfy the project requirements.

Without Machine Learning, all three employees would receive the same ranking.

With Machine Learning, HAF-EPA can make a more intelligent and data-driven allocation decision, similar to how an experienced project manager would evaluate multiple qualified candidates before making a final assignment decision.

2. The way I understand the training is that you match each employee to each past project and tell the tool if that was a good match, so that it then learns that Employee A is usually a good match to a certain kind of project. Correct ?

Answer:

No.

The model does not learn employee names such as Christopher, Donna, or Michael.

Instead, the model learns patterns from employee-project feature vectors and their corresponding suitability labels.

For example, during training the model may observe thousands of employee-project pairs **such as:**

Skill Match = High
Experience = High
Availability = Good
Past Performance = Strong
Label = Suitable (1)

And

Skill Match = Low
Experience = Low
Missing Required Skills = High
Label = Not Suitable (0)

Over time, the Random Forest model learns that certain combinations of features are more likely to result in successful project assignments.

For example, the model may learn patterns such as:

High Skill Match + High Experience + Good Availability = Usually Suitable

or

Many Missing Required Skills + Low Experience = Usually Not Suitable

When a new project arrives, the system first generates a feature vector for each employee.

Example:

Christopher + New Project

↓

[skill_match_score = 1.00, experience = 12.42, availability = 0.30, ...]

The trained model then compares these feature values with the patterns it learned during training and estimates a suitability probability.

Therefore, the model does not think:

⇒ Christopher is usually good.

Instead, it thinks:

⇒ This employee has a feature pattern similar to many previously successful employee-project assignments."

Based on this learned pattern, the model predicts a suitability score and ranks employees for the new project.

3. The model will be trained for a defined set of employees. So companies could not share pre-trained models with each other, right?

Answer

In the context of the current HAF-EPA implementation, the answer is generally **No**.

The model was trained using historical employee-project assignments from a specific workforce dataset. During training, the model learned allocation patterns from employees such as:

Christopher Bass, Donna Jordan , James Collins ... ,and so on.

Together with their:

- Skills
- Experience
- Availability
- Performance characteristics
- Project assignment history

As a result, the learned patterns reflect the **characteristics** of that particular employee population and project environment.

Now consider a different company.

Company B employees: Firoz, Monir , Tareq ... and so on.

These employees may have completely different:

- **Skills**
- **Experience levels**
- **Technologies**
- Project types
- Work allocation patterns
- Workforce characteristics

Therefore, the feature distributions seen by the model may be very different from those used during training.

Because the model has never learned from Company B's historical employee-project assignments, its predictions may not accurately represent the suitability patterns of Company B.

For this reason, a model trained on one company's workforce is generally not expected to achieve optimal performance when directly applied to a completely different company.

The most reliable approach would be to retrain or fine-tune the HAF-EPA model using historical employee-project assignments from the target company.

Therefore, although the model can technically be executed on another company's data, the recommendation quality and prediction reliability may decrease significantly if the employees, projects, and workforce characteristics differ from the training dataset.

For the current HAF-EPA implementation, the model should primarily be considered company-specific and is most effective when applied to employees and projects that are similar to those used during training.

However, We Can Also Say "Yes" in Certain Situations

A pre-trained HAF-EPA model may still be applicable to another company if the employees of the new company have characteristics that are similar to those used during training.

For example, if **Company B** employees have:

- Similar technical skills
- Similar experience levels
- Similar communication abilities
- Similar management capabilities
- Similar work behaviour patterns
- Similar project types

then the employee-project feature vectors generated for Company B will be similar to the feature vectors used to train the model.

In this situation, the learned allocation patterns may still be relevant, and the pre-trained model can potentially produce meaningful recommendations without requiring complete retraining.

This is because the HAF-EPA model does not learn employee names. Instead, it learns relationships between employee-project features and assignment suitability.

Therefore, if the feature patterns of Company B are sufficiently similar to those seen during training, the pre-trained model may still be effective.

However, the greater the difference between the training data and the new company's workforce characteristics, the lower the expected prediction reliability.

Conclusion

The trained HAF-EPA model is generally not intended to be shared as a universal model across completely different companies because the learned allocation patterns depend on the characteristics of the training dataset.

However, the HAF-EPA framework itself can be used by any company.

A company can train a new HAF-EPA model using its own historical employee-project assignments, workforce characteristics, and project data.

As a result, each organization can generate a company-specific model that reflects its own employees, skills, experience profiles, project types, and allocation patterns.

Therefore, while a pre-trained model may only be reliable for organizations with similar workforce characteristics, the HAF-EPA framework can be applied to any company by training a new model using that company's dataset.

4. Can We Apply Transfer Learning in HAF-EPA?

Answer: **Yes**, Transfer learning could be applied as a future enhancement of the HAF-EPA framework.

For example,

a model could first be trained using historical employee-project assignments from Company A. Instead of training a completely new model for Company B, some of the knowledge learned from Company A could be reused and adapted using Company B's employee-project data.

This approach could be beneficial because many workforce characteristics are common across organizations, **such as:**

- Technical skills
- Communication skills
- Management skills
- Experience levels
- Project allocation patterns

For example, if the model has already learned that employees with strong skill compatibility, relevant experience, and good performance records are more likely to be successful project assignments, this knowledge may still be useful in another company.

A transfer learning approach would allow the model to retain general allocation knowledge while adapting to the specific workforce characteristics of Company B through additional training.

This concept is commonly used in Transformer-based models and other deep learning frameworks, where a pre-trained model is fine-tuned on a new dataset rather than being trained from scratch.

However, this is not part of the current HAF-EPA implementation.

The current HAF-EPA framework uses a Random Forest classifier and trains a new model directly using the target company's employee-project dataset.

Therefore, transfer learning and model fine-tuning should be considered as future work for improving model portability across organizations.

5. Could ML identify hidden required skills?

Answer:

No. The current HAF-EPA model cannot identify hidden or new skills automatically. It only learns from the skills and features available in the training data. However, discovering hidden skill relationships could be an interesting topic for future research using NLP, Knowledge Graphs, or LLM-based approaches.

6. Do we need to create a completely new Model every time a new employee or skill appears?

Answer : Not necessarily.

For example, suppose the company already has Backend Developers, Frontend Developers, and Database Engineers, and the HAF-EPA model has already been trained using historical employee-project assignments.

After some time, the company hires a new employee.

In this case, we do not need to create a completely new model. We only need to add the new employee's information, such as skills, experience, availability, and other relevant attributes. The existing HAF-EPA model can then evaluate the new employee and generate recommendations.

However, if the company introduces many new skills, new job roles, or significant changes in project requirements, retraining the model with updated employee-project assignment data may improve prediction quality.

Therefore, a completely new model is not required every time a new employee appears. The existing model can continue to be used, while periodic retraining with newer data helps keep the predictions accurate and up to date.

7. Explainable AI not fully visible !

Answer:

Yes, the UI can be improved to better visualize and explain the recommendation process.

However, it is important to note that the main contribution of this thesis is the HAF-EPA framework itself, including model generation, training, evaluation, and employee-project suitability prediction.

The web application was developed primarily as a demonstration platform to show how the trained HAF-EPA model can be applied in a practical real-world scenario.

Its main purpose is to verify that the trained model can successfully process a new project, generate employee recommendations, and produce prediction results from an end-user perspective.

The Artificial Intelligence component is implemented within the HAF-EPA framework, specifically through:

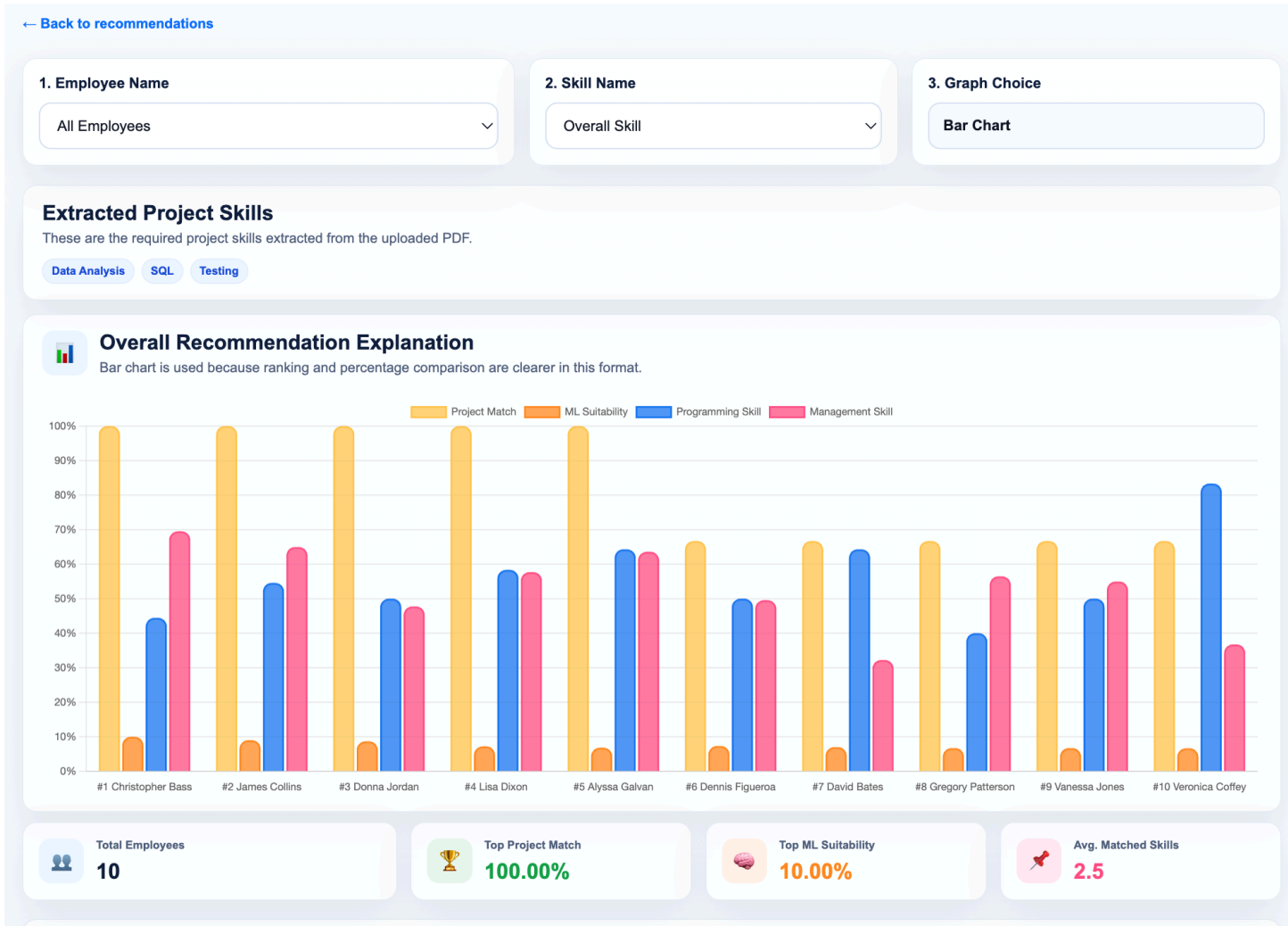
- Employee-project feature engineering
- Machine learning model training
- Suitability prediction
- Employee ranking and recommendation

Therefore, the AI process is already fully executed internally. The framework generates predictions, calculates suitability probabilities, and ranks employees based on the learned allocation patterns from historical employee-project assignments.

Suggestion and Recommendations

1. Keep Only the Bar Chart

Answer:



2. Show Extracted Project Skills

Anser:

Uploaded file: **StockPro.pdf**

Extracted Project Skills

These skills were detected from the uploaded PDF and used for employee-project matching.

- Data Analysis
- SQL
- Testing

View Bar Chart and Explanation

Top 10 Employees

Rank	Employee ID	Name	Project Match %	ML Suitability %	Matched Project Skills	Why Recommended
1	E00087	Christopher Bass	100.00%	10.00%	Data Analysis SQL Testing 3 of 3 extracted skills matched	matched 3 of 3 extracted project skills; project-skill match 100.00%; ML suitability score 10.00%; experience value 12.4; availability 30.00%
2	E00395	James Collins	100.00%	9.00%	Data Analysis SQL Testing 3 of 3 extracted skills matched	matched 3 of 3 extracted project skills; project-skill match 100.00%; ML suitability score 9.00%; experience value 11.4; availability 30.00%
3	E00130	Donna Jordan	100.00%	8.67%	Data Analysis SQL Testing 3 of 3 extracted skills matched	matched 3 of 3 extracted project skills; project-skill match 100.00%; ML suitability score 8.67%; experience value 0.5; availability 60.00%
4	E00119	Lisa Dixon	100.00%	7.22%	Data Analysis SQL Testing 3 of 3 extracted skills matched	matched 3 of 3 extracted project skills; project-skill match 100.00%; ML suitability score 7.22%; experience value 5.4; availability 70.00%
5	E00453	Alyssa Galvan	100.00%	6.83%	Data Analysis SQL Testing 3 of 3 extracted skills matched	matched 3 of 3 extracted project skills; project-skill match 100.00%; ML suitability score 6.83%; experience value 11.1; availability 60.00%